universidade de aveiro

Daniel Corujo
Paulo Salvador
João Miguel Matos

Redes e Serviços

Guidelines de Projetos (v2)

Changelog:
- v2:
  - Correcção da data das apresentações (slide 3)

02/04/2026

PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU

Projetos

- O projeto corresponde a 25% da nota da disciplina
  - 1,5 ECTS
  - 37,5 a 45 horas
  - 7 semanas de aulas para o projeto: 20/4 a a 3/6
  - +/- 6h por semana
- A realização do projeto decorre em paralelo com as aulas teóricas e práticas
  - Por isso essas 37,5 a 45 horas decorrem em regime não-presencial
  - Mas os alunos **podem** e **devem** tirar dúvidas com os professores sempre que precisarem

Avaliação

- Não é preciso relatório
- O vosso trabalho é defendido numa apresentação do vosso grupo, feita na aula prática do dia **28/5**
  - Duração da apresentação ainda por determinar
- Nessa apresentação
  - Alunos terão que explicar o que fizeram, e provar o seu funcionamento
    - (notem que um trabalho que não funciona pode ainda ser um grande trabalho)
  - Cada um dos elementos do grupo terá que demonstrar o que conhece
    - (*no free rides*)
  - Deverão realizar ao vivo capturas de wireshark a demonstrar as interações
  - Deverão ver como o serviço reage em situações de latência (a ver nas próximas aulas práticas)
  - No fim da apresentação, os alunos terão que responder a perguntas dos professores
- Fórmula de avaliação dos projetos: 30% (apresentação)+ 30% (complexidade do trabalho) + 40% (respostas às perguntas)

Escolha de temas

- Para evitar muitos grupos escolherem todos os mesmos trabalhos, cada trabalho vai ter um número máximo de escolhas possível
  - Número ainda a determinar
- Cada grupo vai ter que indicar, por ordem de preferência, os três trabalhos em que está interessado em trabalhar
- A preferência deverá ser expressa neste ficheiro

  RS_TEMAS_26.xlsx
- Os alunos têm até dia 15 de abril às 16h para indicarem as suas escolhas

Temas

Tema 1 – Reverse Proxy & Load Balancer Assincrono

- Construir um serviço que atue como um reverse proxy, gerindo o tráfego entre clientes e múltiplos serviços internos (Docker)
  - O proxy deve receber pedidos HTTP destinados a um de vários containers docker
  - O proxy deve analisar os pedidos (e.g., cabeçalhos como o X-Forwarded-For) e decidir para que container enviar o pedido.
  - Após o grupo conseguir fazer isto, devem implementar um algoritmo de load balancing, e.g. Round Robin, para distribuir a carga. Neste cenário, a carga terá que ser distribuída entre várias instâncias do mesmo tipo de containers (i.e., se houverem 2 containers de servidor web, o load balancing vai enviar os pedidos recebidos para cada um desses servidores, de forma intercalada)
  - O passo seguinte será implementar um mecanismo de prioritização de pedidos. Ou seja, haverão serviços/containers que terão mais prioridade que os outros. O reverse proxy, quando recebe vários pedidos, coloca-os numa fila (queue) e envia primeiro os pedidos dos serviços com maior prioridade
  - Devem usar uma fila assíncrona (asyncio.Queue) onde os pedidos entram e são processados por workers de acordo com a prioridade do endpoint ou do marcador de prioridade definido.
  - Por fim, poderão optimizat o proxy: O proxy deve aceitar ligações externas via HTTP/WebSockets, mas comunicar com os serviços internos via gRPC ou FastAPI (async), demonstrando a conversão de protocolos.

- Implementar funcionalidades de fiabilidade entre um servidor e clients
- Em concreto, deverão implementar controlo de erros, como por exemplo, uma mensagem de ACK enviada pelo servidor a indicar que recebeu o pacote UDP enviado pelo cliente
- Outros mecanismos de controlo de erros: número de sequência (para grandes ficheiros que precisam de ser enviados em vários pacotes), retransmissão de pacotes perdidos
- Mais complexo: Não basta enviar um pacote e esperar. Devem implementar uma janela (à lá TCP) que permita enviar múltiplos pacotes em simultâneo até atingir um limite de "pacotes enviados".
- Devem gerir os timers de retransmissão de forma não bloqueante. Cada pacote enviado deve ter o seu próprio ciclo de vida monitorizado pelo `asyncio`.
- Ainda mais complexo: Implementar Slow Start (controlo de congestão) ou usar a lógica de frames do QUIC para permitir multiplexagem (enviar dois ficheiros diferentes na mesma socket UDP sem que a perda de um bloqueie o outro).

Tema 3 – Serviço de monitoria de Docker com MQTT

- Criar um sistema de monitorização ativa para infraestruturas Docker, focando-se no ciclo de vida dos serviços.
- Cada serviço Docker deve ter um pequeno "agente" (script) que, ao arrancar, publica as suas coordenadas de rede (IP, Porta, ID) num tópico MQTT.
- O agente deve enviar sinais de "presença" periódicos (Heartbeat/Keepalive). Se o broker deixar de receber o sinal de um agente após X segundos, o serviço deve ser marcado como DOWN.
- Devem desenvolver um serviço que subscreva os tópicos de metadados e imprima em tempo real o mapa de saúde da rede de contentores (que contentores existem, quais as coordenadas, se estão down/up, qual o RTT do servidor para eles, etc.)
- O objetivo não é o dado do sensor, mas sim a telemetria do estado da rede e a gestão de disponibilidade dos serviços.

- Implementar um sistema de entrega de conteúdos que minimize a latência e a carga no servidor de origem.
- CDN: rede de servidores espalhados geograficamente com cópias dos ficheiros existentes num servidor original, centralizado. Quando os utilizadores pedem um ficheiro, o pedido é enviado sempre primeiro ao nó de cache da CDN.
- O cliente pede um ficheiro ao nó da CDN. Se o nó tiver o ficheiro (Cache Hit), entrega-o. Se não tiver (Cache Miss), deve descarregá-lo do servidor original, guardá-lo localmente (usando, e.g., `aiofiles`) e só então entregar ao cliente.
- A CDN deve correr num contentor com armazenamento persistente para simular o cache de ficheiros pesados.
- Mais dificil: Quando um ficheiro é alterado no servidor original, este publica uma mensagem "PURGE" num tópico MQTT. O nó da CDN, que está subscrito para receber essas mensagens, recebe o aviso e apaga a sua cópia local imediatamente para evitar servir dados obsoletos.
  - Concorrência: O nó da CDN deve ser capaz de servir múltiplos clientes em simultâneo enquanto descarrega novos ficheiros da origem.

Tema 5 – Jogo Multiplayer Textual

- !!! WARNING !!! Trabalho mais complexo!
- Jogo multiplayer usando texto. Os clientes ligam-se uns aos outros, e enviam mensagens de texto (tipo o trabalho de “Echo” que vão fazer na próxima aula). Só que em vez de repetir no terminal a mensagem recebida (echo), as mensagens têm “comandos” que afetam os jogadores. Exemplo
  - Jogador A envia ao B a mensagem “Atacar com Espada”. O cliente do jogador B ao receber a mensagem, perde 10 pontos de vida
- Problema:
  - Criar um jogo onde não existe um servidor central. Cada jogador é, ao mesmo tempo, cliente e servidor, mantendo a rede viva.
  - Implicação: Se o Jogador A e o Jogador C enviarem concorrentemente mensagens de “Atacar com Espada” ao jogador B, o jogador B perde 10 + 10 pontos de vida
- Devem implementar uma rede descentralizada (Arquitetura P2P) onde os nós se descobrem mutuamente.
- Embora não tenham estudado Kademlia (Kademlia Documentation — Kademlia 2.2.3 documentation), devem aplicar modelos de Redes P2P Estruturadas para garantir que, ao procurar o endereço de um jogador, a rede saiba responder onde ele está.
- Alternativamente a sockets TCP, poderão implementar a interação entre jogadores (atacar, mover, falar) deve ser feita via gRPC.
  - Precisam de definir um ficheiro .proto para serializar estas ações.
- O jogo corre no terminal, mas a rede é assíncrona (gestão de estado e concorrência).
  - Devem usar `aioconsole` para permitir que o utilizador escreva comandos enquanto, em background, o servidor gRPC recebe e processa ataques de outros jogadores sem bloquear a interface.
- Os endereços IP/Portas dos serviços gRPC de cada jogador podem ser armazenados e consultados na DHT (Distributed Hash Table).

HAPPY EASTER BREAK
SANTIGY
FIAFE

universidade de aveiro theoria poiesis praxis